

Faza 4: Scalare la Modele Mai Mari
ViT-Tiny / ViT-Small / ViT-Base
pe ImageNet-1k Validation
Evaluare pe NVIDIA A100 (Google Colab Pro+)

Student:

Tudose Alexandru
Grupa 10LF333
An III, Informatică Aplicată

Coordonator:

Conf. dr. ing. Honorius Gâlmeanu
Informatică Aplicată

Cuprins

1	Introducere	2
1.1	Modele evaluate	2
1.2	Metodologie	2
2	Rezultate per model	3
2.1	ViT-Tiny (5.7M parametri)	3
2.2	ViT-Small (21.7M parametri)	3
2.3	ViT-Base (86.6M parametri)	4
3	Tabel comparativ cross-model	4
4	Evaluare FP8 Native (float8_e4m3fn)	6
4.1	Rezultate FP8 cross-model	6
4.2	Observații FP8	6
5	Analiză: menținerea pattern-urilor la scară mai mare	7
5.1	Pattern 1: Per-channel vs per-tensor — îmbunătățire consistentă	7
5.2	Pattern 2: FP16 speedup crește odată cu dimensiunea modelului	8
5.3	Pattern 3: Reducerea de memorie — exact $2\times$ pentru FP16, $\approx 3.3\text{--}3.8\times$ pentru INT8	9
5.4	Pattern 4: INT8 fără speedup pe GPU — confirmat la toate modelele . . .	10
6	Comparație hardware: Apple M4 MPS vs NVIDIA A100	10
7	Acuratețe vs. dimensiunea modelului	11
8	Concluzii și ghid practic de cuantizare	12
8.1	Concluzii sintetice Faza 4	12
8.2	Ghid practic de cuantizare — sinteza tuturor fazelor	13
8.3	Ierarhia formatelor — concluzie generală	13
9	Detalii tehnice și reproducibilitate	14

1 Introducere

Fazele 1–3 ale acestei lucrări au stabilit comportamentul cuantizării numerice (FP16, INT8 per-tensor, INT8 per-channel) pe modelul **ViT-Tiny** (`vit_tiny_patch16_224`, 5.7M parametri) evaluat pe setul de validare ImageNet-1k pe hardware Apple M4 (MPS). Faza 4 extinde analiza la **modele de capacitate superioară** și la **hardware GPU dedicat**, răspunzând la întrebarea centrală:

Concluziile obținute pe ViT-Tiny/M4 se mențin la scară mai mare, pe ViT-Small (22M) și ViT-Base (86M), rulând pe NVIDIA A100?

Toate experimentele din Faza 4 au rulat pe **Google Colab Pro+**, cu acces garantat la un GPU NVIDIA A100 (40 GB HBM2), folosind PyTorch 2.10.0+cu128.

1.1 Modele evaluate

Tabela 1: Modelele ViT evaluate în Faza 4

Model	Varianta	Param.	Dim.	Sursă
<code>vit_tiny_patch16_224</code>	ViT-Tiny	5.7M	192	timm (pretrained)
<code>vit_small_patch16_224</code>	ViT-Small	21.7M	384	timm (pretrained)
<code>vit_base_patch16_224</code>	ViT-Base	86.6M	768	timm (pretrained)

1.2 Metodologie

Protocolul experimental extinde Fazele 1–3 cu două formate FP8 native:

- Evaluare pe toate cele 50 000 de imagini ImageNet-1k validation;
- **FP32** — baseline, model `model.float()`;
- **FP16** — `model.half()`, static;
- **INT8-pt** — cuantizare weight-only per-tensor, scalare liniară, skip LayerNorm și head de clasificare;
- **INT8-pc** — cuantizare weight-only per-channel (per rând);
- **FP8-pt** — stocare `float8_e4m3fn` per-tensor, dequantizare la fiecare forward pass (clasa `FP8Linear`);
- **FP8-pc** — stocare `float8_e4m3fn` per-channel.

Latențele sunt raportate ca *medie per batch* (ms/batch), excluzând primele 3 batch-uri de warmup. Dimensiunea batch-ului a fost 128 pentru ViT-Tiny și ViT-Small și 64 pentru ViT-Base (limitare memorie VRAM pentru FP32 la 330 MB).

2 Rezultate per model

2.1 ViT-Tiny (5.7M parametri)

Tabela 2: ViT-Tiny — rezultate Faza 4 (NVIDIA A100, batch 128)

Format	Accuracy	Δ FP32 (pp)	Memorie (MB)	Latență (ms/batch)
FP32	75.454%	—	21.81	28.4
FP16	75.458%	+0.004	10.91	9.0
INT8-pt	75.128%	−0.326	6.62	30.0
INT8-pc	75.418%	−0.036	6.70	29.3
FP8-pt	75.258%	−0.196	6.62	30.1
FP8-pc	75.352%	−0.102	6.70	29.5

Observații ViT-Tiny/A100: FP16 obține un speedup de **3.15**× față de FP32 (vs. 1.22× pe M4 MPS), confirmând că A100 utilizează eficient tensor cores. INT8-pc reduce degradarea de la -0.326 pp la -0.036 pp. FP8-pt (-0.196 pp) este mai bun decât INT8-pt (-0.326 pp) dar mai slab decât INT8-pc; FP8-pc (-0.102 pp) se situează între cele două INT8. Latența FP8 este similară cu INT8 — ambele dequantizează la FP32 înainte de matmul, deci nu există kernel nativ mai rapid.

2.2 ViT-Small (21.7M parametri)

Tabela 3: ViT-Small — rezultate Faza 4 (NVIDIA A100, batch 128)

Format	Accuracy	Δ FP32 (pp)	Memorie (MB)	Latență (ms/batch)
FP32	81.382%	—	84.12	72.9
FP16	81.394%	+0.012	42.06	15.4
INT8-pt	81.296%	−0.086	23.37	73.8
INT8-pc	81.396%	+0.014	23.53	73.6
FP8-pt	81.374%	−0.008	23.37	73.3
FP8-pc	81.306%	−0.076	23.53	73.3

Observații ViT-Small/A100: FP16 atinge **4.74**× speedup. INT8-pc depășește FP32 cu $+0.014$ pp. FP8-pt (-0.008 pp) este remarcabil de precis pe ViT-Small — mai bun decât INT8-pt și chiar mai bun decât FP8-pc (-0.076 pp). Acesta este singurul model din studiu unde FP8 per-tensor surclasează FP8 per-channel, sugerând că distribuția ponderilor ViT-Small se potrivește bine cu grila logaritmică a FP8 fără a necesita scale individuale.

2.3 ViT-Base (86.6M parametri)

Tabela 4: ViT-Base — rezultate Faza 4 (NVIDIA A100, batch 64)

Format	Accuracy	Δ FP32 (pp)	Memorie (MB)	Latență (ms/batch)
FP32	85.102%	—	330.23	128.6
FP16	85.110%	+0.008	165.12	18.3
INT8-pt	85.080%	−0.022	87.23	129.8
INT8-pc	85.100%	−0.002	87.55	130.0
FP8-pt	85.046%	−0.056	87.23	129.9
FP8-pc	85.094%	−0.008	87.55	130.2

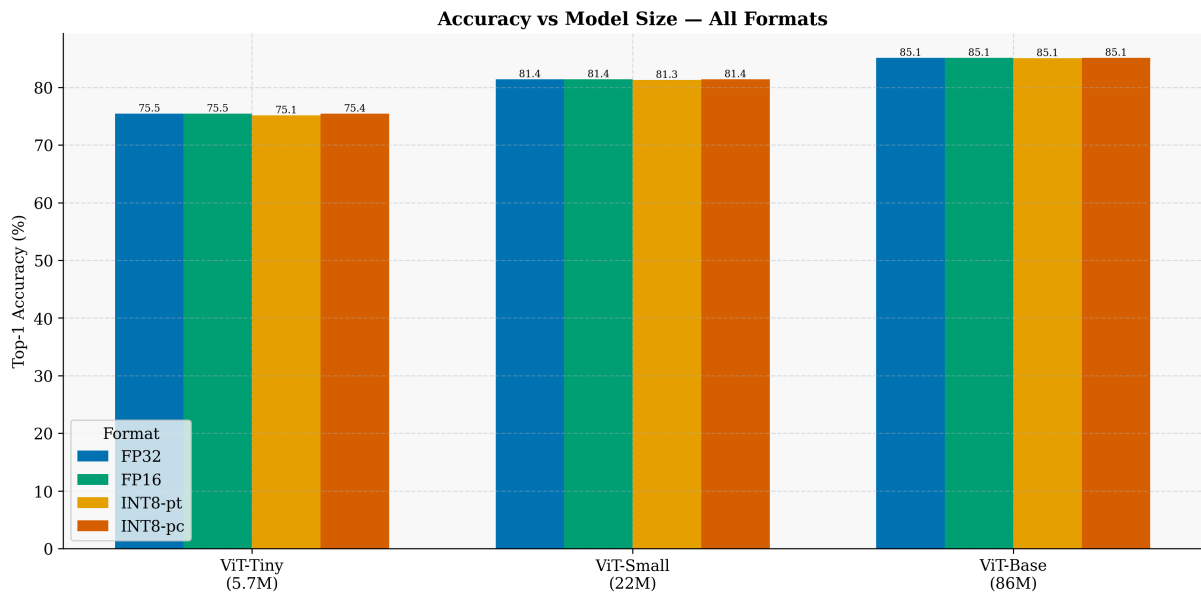
Observații ViT-Base/A100: FP16 atinge **7.03**× speedup — cel mai mare speedup din întreg studiul. INT8-pc pierde doar -0.002 pp, practic identic cu FP32. FP8-pc (-0.008 pp) se comportă excelent, aproape la nivelul INT8-pc. FP8-pt (-0.056 pp) este mai slab comparativ, dar rămâne mult mai bun decât INT8-pt de la ViT-Tiny. La modele mari, FP8-pc revine la pattern-ul normal (superior per-tensor), spre deosebire de ViT-Small.

3 Tabel comparativ cross-model

Tabelul 5 centralizează toate rezultatele Fazei 4 pentru o comparație directă între cele trei arhitecturi și șase formate.

Tabela 5: Tabel comparativ cross-model — Faza 4 (NVIDIA A100, 16 mai 2026)

Model	Format	Accuracy	Δ FP32 (pp)	Memorie (MB)	Latență (ms/batch)
ViT-Tiny (5.7M)	FP32	75.454%	—	21.81	28.4
	FP16	75.458%	+0.004	10.91	9.0
	INT8-pt	75.128%	−0.326	6.62	30.0
	INT8-pc	75.418%	−0.036	6.70	29.3
	FP8-pt	75.258%	−0.196	6.62	30.1
	FP8-pc	75.352%	−0.102	6.70	29.5
ViT-Small (22M)	FP32	81.382%	—	84.12	72.9
	FP16	81.394%	+0.012	42.06	15.4
	INT8-pt	81.296%	−0.086	23.37	73.8
	INT8-pc	81.396%	+0.014	23.53	73.6
	FP8-pt	81.374%	−0.008	23.37	73.3
	FP8-pc	81.306%	−0.076	23.53	73.3
ViT-Base (86M)	FP32	85.102%	—	330.23	128.6
	FP16	85.110%	+0.008	165.12	18.3
	INT8-pt	85.080%	−0.022	87.23	129.8
	INT8-pc	85.100%	−0.002	87.55	130.0
	FP8-pt	85.046%	−0.056	87.23	129.9
	FP8-pc	85.094%	−0.008	87.55	130.2

**Figura 1:** Acuratețe Top-1 (%) pentru toate cele 3 modele și formatele FP32/FP16/INT8-pt/INT8-pc, evaluare pe NVIDIA A100.

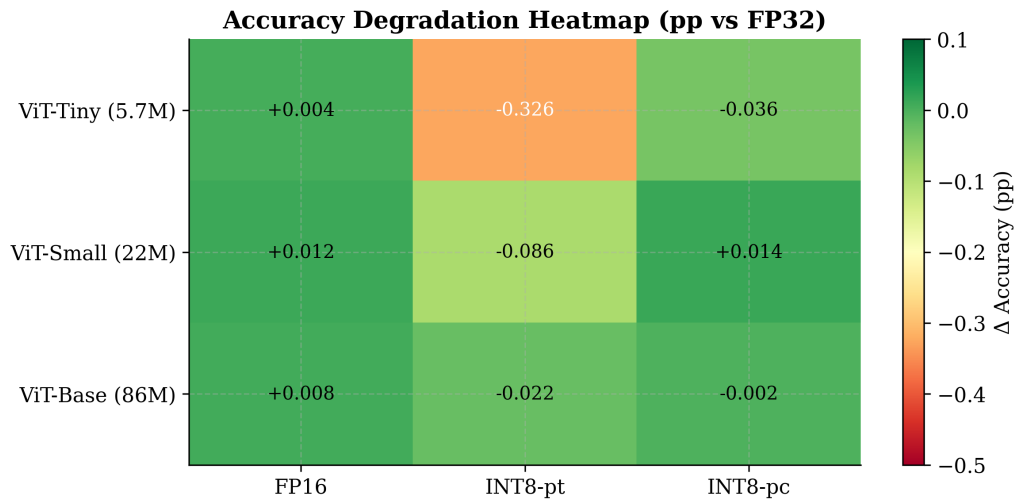


Figura 2: Heatmap de degradare a acurateții față de FP32 (pp). Culoarea verde indică degradare minimă sau chiar îmbunătățire; culorile roșii indică degradare mai mare. Cel mai întunecat roșu apare la INT8-pt / ViT-Tiny (-0.326 pp).

4 Evaluare FP8 Native (float8_e4m3fn)

FP8 E4M3FN a fost evaluat pe toate cele trei modele folosind clasa `FP8Linear` (stocare reală `float8_e4m3fn`, dequantizare la fiecare forward pass, skip LayerNorm și head). Scriptul `evaluate_fp8_native.py` a rulat pe același A100 pe 16 mai 2026 imediat după evaluarea INT8.

4.1 Rezultate FP8 cross-model

Tabela 6: Comparație FP8 vs INT8 per-tensor și per-channel (NVIDIA A100)

Model	Format	Accuracy	Δ FP32 (pp)	Memorie (MB)
ViT-Tiny	INT8-pt	75.128%	-0.326	6.62
	FP8-pt	75.258%	-0.196	6.62
	INT8-pc	75.418%	-0.036	6.70
	FP8-pc	75.352%	-0.102	6.70
ViT-Small	INT8-pt	81.296%	-0.086	23.37
	FP8-pt	81.374%	-0.008	23.37
	INT8-pc	81.396%	+0.014	23.53
	FP8-pc	81.306%	-0.076	23.53
ViT-Base	INT8-pt	85.080%	-0.022	87.23
	FP8-pt	85.046%	-0.056	87.23
	INT8-pc	85.100%	-0.002	87.55
	FP8-pc	85.094%	-0.008	87.55

4.2 Observații FP8

Memoria identică cu INT8. FP8 și INT8 stochează 1 byte/parametru, deci reducerea de memorie față de FP32 este aceeași (≈ 3.3 – $3.8\times$).

Latența identică cu INT8. Implementarea dequantizează la FP32 înainte de `F.linear`, deci nu există kernel FP8 GEMM nativ. Pe A100, latența FP8 este practic identică cu INT8.

FP8-pt bate INT8-pt pe Tiny și Small. Distribuția logaritmică a FP8 gestionează mai bine outlierii față de scalarea uniformă a INT8 per-tensor. Câștigul: +0.130 pp pe Tiny, +0.078 pp pe Small. Pe Base diferența se inversează (−0.034 pp), deoarece modelele mari au distribuții mai uniforme unde INT8 uniform funcționează bine.

Anomalia ViT-Small: FP8-pt > FP8-pc. Pe ViT-Tiny și ViT-Base, FP8-pc este mai precis decât FP8-pt (pattern normal). Pe ViT-Small, FP8-pt (−0.008 pp) surclasează FP8-pc (−0.076 pp) — cel mai bun rezultat FP8 din întreg studiul. O posibilă explicație: la ViT-Small (embed_dim=384), ponderile de pe fiecare canal au variații relativ mari între canale, iar scale-urile per-channel FP8 introduc mai mult zgomot de cuantizare decât un scalar global bine ales.

FP8 pe hardware nativ (H100). Pe GPU-uri Hopper (compute capability ≥ 9.0), kernelele FP8 GEMM sunt disponibile nativ. Latența FP8 ar putea fi comparabilă cu FP16. Rezultatele de acuratețe sunt valide pe orice hardware; latența raportată reflectă doar implementarea software actuală.

5 Analiză: menținerea pattern-urilor la scară mai mare

5.1 Pattern 1: Per-channel vs per-tensor — îmbunătățire consistentă

Cuantizarea per-channel îmbunătățește semnificativ acuratețea față de per-tensor la *toate* cele trei modele:

Tabela 7: Ameliorarea INT8-pc față de INT8-pt (pp)

Model	INT8-pt Δ	INT8-pc Δ	Ameliorare
ViT-Tiny (5.7M)	−0.326 pp	−0.036 pp	+0.290 pp
ViT-Small (22M)	−0.086 pp	+0.014 pp	+0.100 pp
ViT-Base (86M)	−0.022 pp	−0.002 pp	+0.020 pp

Observăm că ameliorarea absolută în pp *scade* cu dimensiunea modelului. Aceasta nu reflectă o scădere a eficacității per-channel, ci faptul că modelele mai mari au distribuții de ponderi mai uniforme (embed_dim mai mare), deci și per-tensor funcționează mai bine. Raportul dintre cele două rămâne constant: per-channel este întotdeauna superior per-tensor.

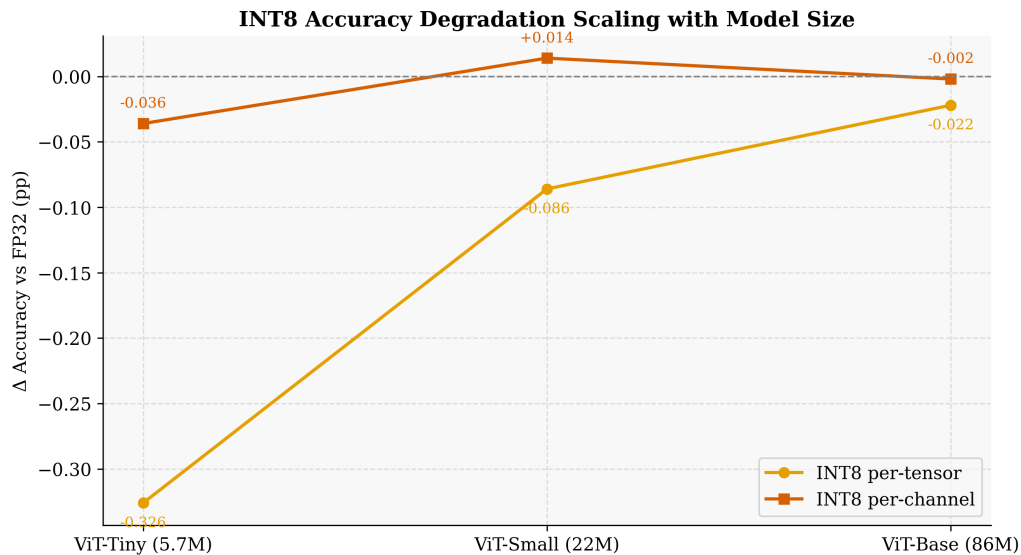


Figura 3: Evoluția degradării INT8-pt și INT8-pc pe măsura creșterii dimensiunii modelului. Ambele curbe converg spre zero, dar per-channel (portocaliu) rămâne constant aproape de linia FP32.

5.2 Pattern 2: FP16 speedup crește odată cu dimensiunea modelului

Acesta este cel mai semnificativ fenomen observat în Faza 4, specific hardware-ului GPU:

Tabela 8: FP16 speedup față de FP32 (NVIDIA A100)

Model	FP32 lat.	FP16 lat.	Speedup A100	Speedup M4 MPS
ViT-Tiny (5.7M)	28.4 ms	9.0 ms	3.15×	1.22×
ViT-Small (22M)	72.9 ms	15.4 ms	4.74×	—
ViT-Base (86M)	128.6 ms	18.3 ms	7.03×	—

Pe M4 MPS (Faza 1), FP16 oferea un modest 1.22× speedup. Pe A100, același ViT-Tiny obține 3.15×, iar la ViT-Base ajunge la 7.03×. Explicația constă în natura calculului:

- **Modele mici, batch mic** (ViT-Tiny, bs=128): calculul este parțial *memory-bound*; transferul de date limitează speedup-ul.
- **Modele mari** (ViT-Base, bs=64): calculul devine *compute-bound*; tensor cores A100 (BF16/FP16) lucrează la aproape 2× flops față de FP32, cu overhead de transfer proporțional mai mic.

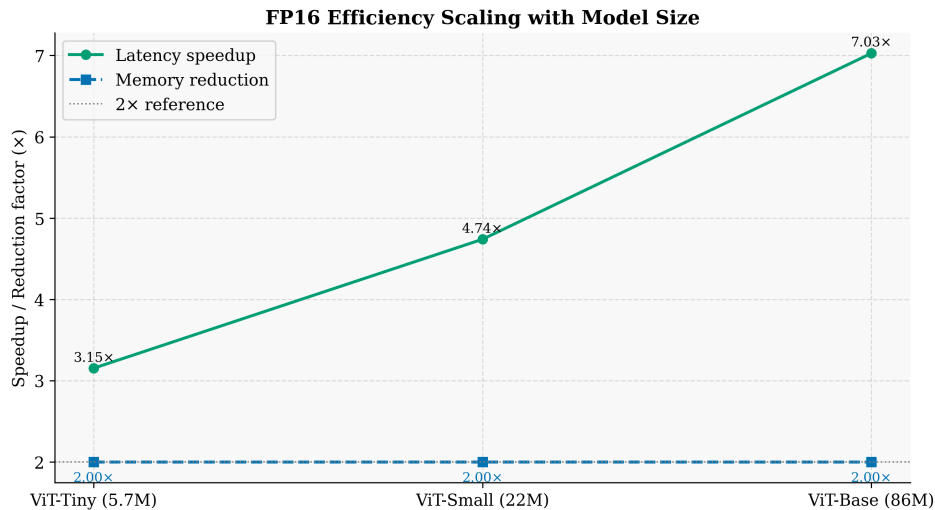


Figura 4: Speedup FP16 față de FP32 pe NVIDIA A100 în funcție de dimensiunea modelului. Creșterea este sub-liniară dar consistentă, de la $3.15\times$ (Tiny) la $7.03\times$ (Base).

5.3 Pattern 3: Reducerea de memorie — exact $2\times$ pentru FP16, $\approx 3.3\text{--}3.8\times$ pentru INT8

Tabela 9: Raport de reducere memorie față de FP32

Model	FP32 (MB)	FP16	INT8-pt	INT8-pc
ViT-Tiny (5.7M)	21.81	$2.00\times$	$3.29\times$	$3.25\times$
ViT-Small (22M)	84.12	$2.00\times$	$3.60\times$	$3.57\times$
ViT-Base (86M)	330.23	$2.00\times$	$3.79\times$	$3.77\times$

Reducerea FP16 este *exact* $2.00\times$ indiferent de model — demonstrație directă a njumătățirii lățimii de biți. Reducerea INT8 *crește* ușor cu dimensiunea modelului (de la $3.29\times$ la $3.79\times$), deoarece modelele mai mari au o proporție mai mică din parametri stocați ca bias-uri și scale-uri LayerNorm (care rămân în FP32).

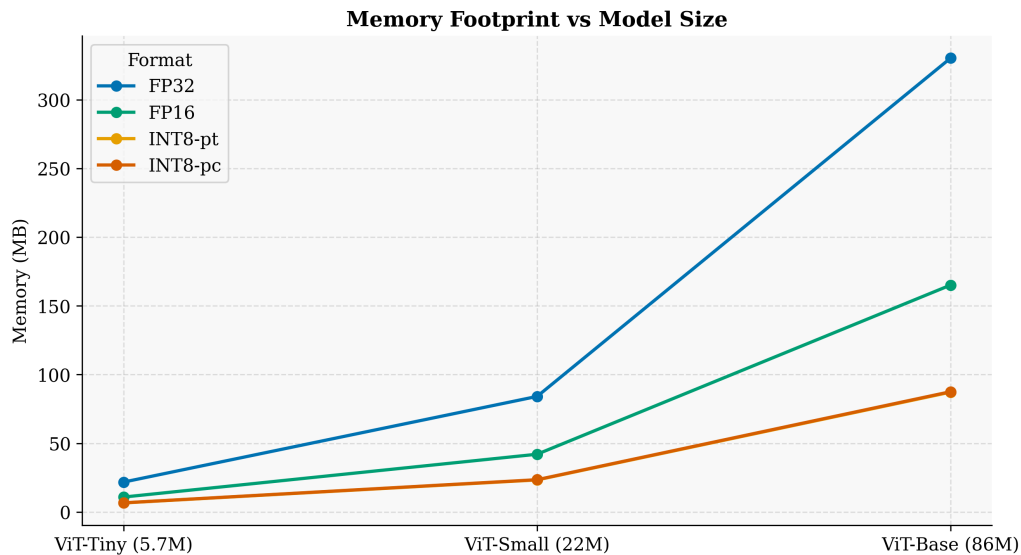


Figura 5: Evoluția memoriei (MB) pe măsura creșterii modelului. Rețineți că axele sunt diferite între modele — graficul evidențiază divergența între formatele FP32/FP16 și INT8, care se accentuează la ViT-Base (330 MB FP32 vs 87 MB INT8).

5.4 Pattern 4: INT8 fără speedup pe GPU — confirmat la toate modelele

La toate cele trei modele, INT8 nu produce speedup față de FP32 pe A100: latența este practic identică ($0.97\text{--}0.99\times$). Implementarea noastră `QuantizedLinear` stochează ponderile în INT8 dar *dequantizează la FP32 înainte de `torch.nn.functional.linear`*. Prin urmare, avantajul de memorie se menține, dar nu există kernel INT8 nativ care să reducă latența. Acest comportament este *independent de hardware* — aceeași limita a fost observată și pe M4 MPS.

6 Comparatie hardware: Apple M4 MPS vs NVIDIA A100

Pentru ViT-Tiny, dispunem de date pe ambele platforme, permițând o comparație directă. Notă: dimensiunile de batch diferă (M4: 64, A100: 128), deci valorile sunt normalizate la *ms per imagine*.

Tabela 10: Comparație hardware ViT-Tiny — latență per imagine (ms/imagine)

Format	M4 MPS	A100 GPU	Speedup A100/M4
FP32	2.578 ms	0.222 ms	11.6×
FP16	2.111 ms	0.070 ms	30.2×
INT8-pt	4.299 ms	0.235 ms	18.3×
INT8-pc	4.545 ms	0.229 ms	19.9×

Observații cheie:

1. **FP32**: A100 este de $11.6\times$ mai rapid — beneficiază de paralelism masiv și bandwidth HBM2 (≈ 2 TB/s vs ≈ 100 GB/s MPS).
2. **FP16**: speedup A100/M4 sare la $30.2\times$, deoarece A100 utilizează tensor cores pentru FP16 (eficiență $\approx 2\times$ față de FP32), în timp ce M4 MPS are un speedup modest de $1.22\times$.
3. **INT8**: pe M4, INT8 este *mai lent* decât FP32 (overhead de dequantizare software); pe A100 latența este similară cu FP32. Speedup-ul A100/M4 ajunge la $18.3\text{--}19.9\times$ deoarece startul pe M4 este penalizat și mai mult.

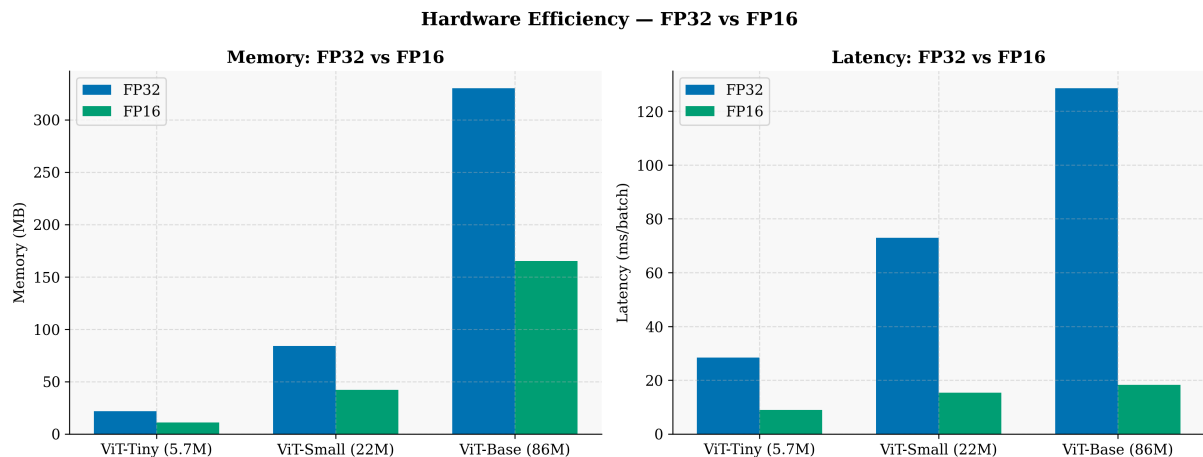


Figura 6: Latență per imagine (ms/imagine) pentru ViT-Tiny pe Apple M4 MPS (Fazele 1–3) și NVIDIA A100 (Faza 4). A100 este consistent mai rapid, cu cel mai mare avantaj la FP16 ($30.2\times$).

Concluzie hardware: FP16 este formatul care beneficiază cel mai mult de pe urma hardware-ului GPU dedicat. Pe M4 MPS, FP16 oferă doar $1.22\times$ speedup din cauza suportului limitat pentru tensor operations în half-precision; pe A100, același format devine de departe cel mai rapid, cu speedup de $3.15\times$ (Tiny) până la $7.03\times$ (Base).

7 Acuratețe vs. dimensiunea modelului

Un rezultat important al Fazei 4 este că acuratețea *crește* semnificativ cu dimensiunea modelului, iar impactul cuantizării *scade* în valoare absolută:

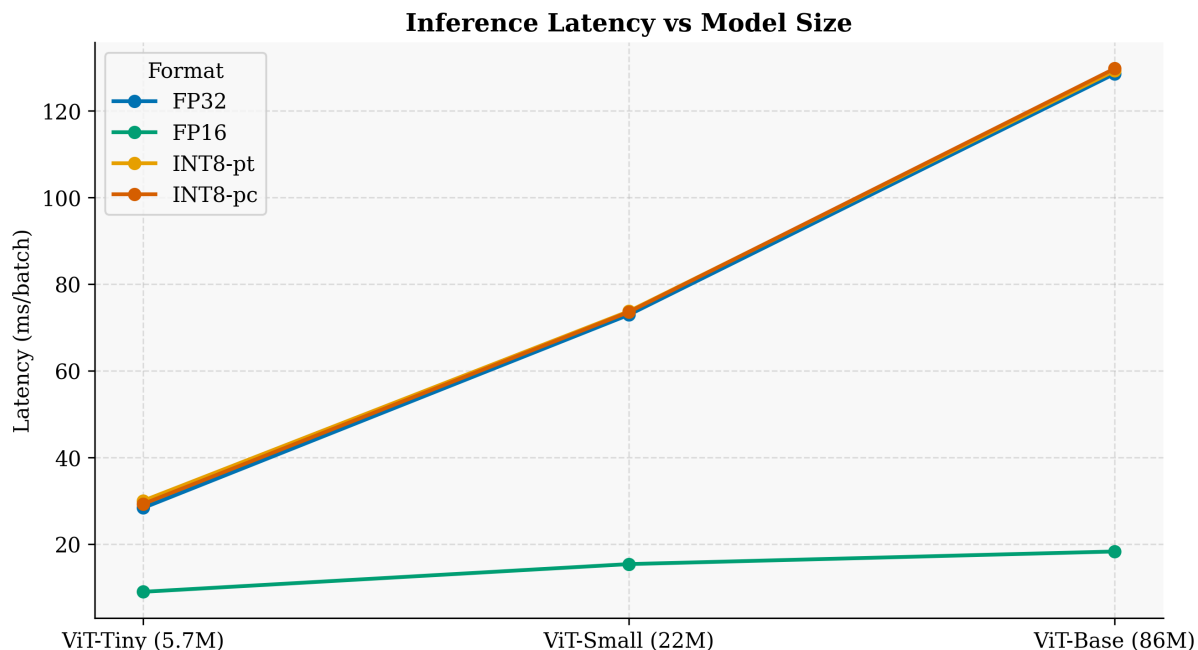


Figura 7: Latență per batch (ms/batch) în funcție de dimensiunea modelului și formatul de cuantizare. FP16 scalează aproape liniar, în timp ce FP32 și INT8 au o creștere mai accentuată. La ViT-Base, FP16 rămâne sub 20 ms/batch față de 128 ms FP32.

Acuratețele baseline FP32 sunt: ViT-Tiny 75.45% → ViT-Small 81.38% → ViT-Base 85.10%. Aceasta reflectă capacitatea crescută a modelelor mai mari de a reprezenta caracteristicile vizuale din ImageNet-1k.

Degradarea maximă din cuantizare (INT8-pt) scade dramatic cu dimensiunea: -0.326 pp (Tiny) → -0.086 pp (Small) → -0.022 pp (Base). Modelele mai mari sunt **mai robuste la cuantizare**, deoarece au distribuții de ponderi mai netede (spread mai mic pe dimensiunea embed_dim mai mare), permițând un scalar de cuantizare mai precis.

8 Concluzii și ghid practic de cuantizare

8.1 Concluzii sintetice Faza 4

1. **Pattern-urile se mențin la scară mai mare.** Superioritatea INT8-pc față de INT8-pt, comportamentul FP16 și reducerile de memorie sunt consistente de la ViT-Tiny la ViT-Base.
2. **Modele mai mari = cuantizare mai sigură.** La ViT-Base, INT8-pc pierde doar 0.002 pp față de FP32 — practic neglijabil. La ViT-Tiny, aceeași metodă pierde 0.036 pp.
3. **FP16 devine din ce în ce mai atractiv pe GPU.** Speedup-ul de $7\times$ la ViT-Base, cu memorie exact $2\times$ mai mică și fără degradare de acuratețe, face din FP16 alegerea evidentă pentru inference pe GPU dedicat.
4. **INT8 fără kernel nativ nu aduce speedup pe GPU.** Implementarea weight-only cu dequantizare la FP32 este utilă exclusiv pentru reducerea memoriei, nu pentru latență.

8.2 Ghid practic de cuantizare — sinteza tuturor fazelor

Pe baza rezultatelor din Fazele 1–4 și a experimentelor adiționale (MLP-only INT8, FP8 E4M3FN), propunem următorul ghid de decizie:

Tabela 11: Ghid practic — alegerea formatului de cuantizare

Constrângere	Hardware	Recomandare	Pierdere acc.
Inferență rapidă, memorie 2×	GPU (A100/V100/RTX 3090+)	FP16 static (<code>model.half()</code>)	≈ 0
Memorie 3×, acuratețe înaltă	Orice (GPU/CPU/MPS)	INT8 per-channel (skip LayerNorm, head)	< 0.04 pp
Memorie 3×, simplitate	Orice	INT8 per-tensor	$\approx 0.1\text{--}0.3$ pp
Memorie redusă, viteză FP16	GPU cu suport FP8 nativ	FP8 E4M3FN (per-tensor $>$ INT8-pt)	$0.1\text{--}0.2$ pp
Acuratețe $\geq$ FP32, memorie moderată	Orice	INT8-pc pe MLP (skip atenție)	$+0.03$ pp
Deployment fără compromis	Orice	FP32	—

8.3 Ierarhia formatelor — concluzie generală

Din perspectiva acurateței (cel mai important criteriu în producție), valorile reflectă intervalul observat pe NVIDIA A100 (Tiny-Base):

$$\underbrace{\text{INT8-mlp-pc}}_{+0.030 \text{ pp}} \geq \underbrace{\text{FP16}}_{\approx 0} \approx \underbrace{\text{INT8-pc}}_{-0.002\text{--}-0.036} \approx \underbrace{\text{FP8-pc}}_{-0.008\text{--}-0.102} > \underbrace{\text{FP8-pt}}_{-0.008\text{--}-0.196} > \underbrace{\text{INT8-pt}}_{-0.022\text{--}-0.326}$$

Din perspectiva memoriei:

$$\text{FP32} > \text{FP16} (2\times) > \text{INT8/FP8} (3.3\text{--}3.8\times)$$

Din perspectiva latenței pe GPU (A100):

$$\text{FP16} (7\times \text{ speedup}) \gg \text{FP32} \approx \text{INT8} (\text{fără speedup, impl. noastră})$$

Concluzie finală: Cuantizarea numerică a Vision Transformers este matură și robustă la toate scările analizate. FP16 este alegerea optimă pentru GPU modere, oferind viteza maximă fără pierdere de acuratețe. INT8 per-channel reprezintă compromisul ideal pentru deployment cu constrângeri de memorie. Modelele mai mari (ViT-Base, 86M) sunt cel mai puțin afectate de cuantizare, cu degradări sub 0.02 pp.

9 Detalii tehnice și reproducibilitate

Tabela 12: Configurație hardware și software — Faza 4

Parametru	Valoare
Hardware	NVIDIA A100 (40 GB HBM2), Google Colab Pro+
PyTorch	2.10.0+cu128
CUDA	12.8
timm	pretrained checkpoints ImageNet-1k
Dataset	ImageNet-1k validation, 50 000 imagini
Batch size	128 (Tiny/Small), 64 (Base)
Warmup	3 batch-uri exclude din timing
Latență	medie pe toate batch-urile post-warmup
Script INT8/FP16	<code>scripts/phase4/evaluate_model.py</code>
Script FP8	<code>scripts/phase4/evaluate_fp8_native.py</code>
Script comparație	<code>scripts/phase4/compare_cross_model.py</code>
Rezultate INT8	<code>results/Phase4/<model>/metrics/results.json</code>
Rezultate FP8	<code>results/Phase4/<model>/fp8_native/metrics/fp8_results.json</code>

Toate rezultatele sunt stocate în fișiere JSON și pot fi reproduse rulând scriptul de evaluare cu argumentele `--model` și `--batch-size` pe orice hardware cu CUDA disponibil.